

```

1  import javax.jms.Connection;
2  import javax.jms.ConnectionFactory;
3  import javax.jms.Destination;
4  import javax.jms.Message;
5  import javax.jms.MessageConsumer;
6  import javax.jms.MessageListener;
7  import javax.jms.MessageProducer;
8  import javax.jms.Session;
9  import javax.jms.TextMessage;
10 import org.apache.activemq.ActiveMQConnection;
11 import org.apache.activemq.ActiveMQConnectionFactory;
12
13 public class HelloTopic {
14     private static String url = ActiveMQConnection.DEFAULT_BROKER_URL;
15     private static String subject = "MyTopic";
16
17     public static void main(String argv[]) {
18         try {
19             ConnectionFactory connectionFactory = new ActiveMQConnectionFactory(url);
20             Connection connection = connectionFactory.createConnection();
21             connection.start();
22             Session session = connection.createSession(false, Session.AUTO_ACKNOWLEDGE);
23             Destination destination = session.createTopic(subject);
24             MessageProducer producer = session.createProducer(destination);
25             MessageConsumer consumer = session.createConsumer(destination);
26             MessageListener listener =
27                 new MessageListener() {
28                     public void onMessage(Message msg) {
29                         try {
30                             TextMessage textmsg = (TextMessage) msg;
31                             System.out.println("received : " + textmsg.getText());
32                         } catch (Exception ex) {
33                             ex.printStackTrace();
34                         }
35                     }
36                 };
37             consumer.setMessageListener(listener);
38             TextMessage message = session.createTextMessage("Hello welcome to ActiveMQ!");
39             producer.send(message);
40             System.out.println("Sentage '" + message.getText() + "'");
41             Thread.sleep(1000);
42             connection.close();
43         } catch (Exception ex) {
44             ex.printStackTrace();
45             return;
46         }
47     }
48 }
49
50 import java.awt.Button;
51 import java.awt.Color;
52 import java.awt.FlowLayout;
53 import java.awt.Frame;

```

```

54 import java.awt.TextArea;
55 import java.awt.TextField;
56 import java.awt.event.ActionEvent;
57 import java.awt.event.ActionListener;
58 import java.util.Vector;
59 import javax.jms.Connection;
60 import javax.jms.ConnectionFactory;
61 import javax.jms.MessageConsumer;
62 import javax.jms.MessageProducer;
63 import javax.jms.Session;
64 import javax.jms.Topic;
65 import org.apache.activemq.ActiveMQConnection;
66 import org.apache.activemq.ActiveMQConnectionFactory;
67 import javax.jms.MessageListener;
68 import javax.jms.TextMessage;
69 import javax.jms.Message;
70 import java.util.Collections;
71
72 public class Irc {
73     public static TextArea text;
74     public static TextField data;
75     public static Frame frame;
76     public static String url = ActiveMQConnection.DEFAULT_BROKER_URL;
77     public static String subject = "MyQueue";
78     public static Vector<String> users = new Vector<String>();
79     public static String myName;
80     public static ConnectionFactory connectionFactory;
81     public static Connection connection;
82     public static Session session;
83     public static MessageConsumer consumer;
84     public static MessageProducer producer;
85     public static Topic topic;
86
87     public static void main(String argv[]) {
88         if (argv.length != 1) {
89             System.out.println("java Irc <name>");
90             return;
91         }
92         myName = argv[0];
93         frame = new Frame();
94         frame.setLayout(new FlowLayout());
95         text = new TextArea(10, 55);
96         text.setEditable(false);
97         text.setForeground(Color.red);
98         frame.add(text);
99         data = new TextField(55);
100        frame.add(data);
101        Button write_button = new Button("write");
102        write_button.addActionListener(new writeListener());
103        frame.add(write_button);
104        Button connect_button = new Button("connect");
105        connect_button.addActionListener(new connectListener());
106        frame.add(connect_button);
107        Button who_button = new Button("who");

```



```

162         IRC.print("--- User " + user + " disconnected ! ---");
163     } else {
164         IRC.print(stringMsg);
165     }
166     } catch (Exception ex) {
167         ex.printStackTrace();
168     }
169 }
170 };
171 IRC.consumer.setMessageListener(listener);
172 TextMessage message = IRC.session.createTextMessage("#connect: "
173                                                     + IRC.myName);
174 IRC.producer.send(message);
175 System.out.println("Connected to "+ IRC.subject);
176 } catch (Exception ex) {
177     ex.printStackTrace();
178 }
179 }
180 }
181
182 class whoListener implements ActionListener {
183     public void actionPerformed(ActionEvent ae) {
184         try {
185             IRC.print("Users : " + String.join(",", IRC.users));
186         } catch (Exception ex) {
187             ex.printStackTrace();
188         }
189     }
190 }
191
192 class leaveListener implements ActionListener {
193     public void actionPerformed(ActionEvent ae) {
194         try {
195             TextMessage message = IRC.session.createTextMessage("#disconnect: "
196                                                     + IRC.myName);
197             IRC.producer.send(message);
198             IRC.producer.close();
199             IRC.consumer.close();
200             IRC.session.close();
201             IRC.connection.close();
202             System.out.println("Connection close");
203         } catch (Exception ex) {
204             ex.printStackTrace();
205         }
206     }
207 }

```